

Guia 3 — Staging Area e Commits Avançados

Este guia aprofunda o funcionamento da **staging area**, explica como o Git registra mudanças e apresenta comandos essenciais para trabalhar com commits de forma mais avançada.

1. O que é a Staging Area (de verdade)

A staging area é uma área intermediária entre:

- **working directory** → onde seus arquivos vivem e são modificados
- **repositório Git** → onde ficam os commits já registrados

Ela funciona como uma bandeja de supermercado:

- você coloca itens nela (git add)
- depois confirma tudo de uma vez no caixa (git commit)

Essa etapa intermediária existe para dar controle total sobre o que vai entrar no commit.

Com ela, você pode:

- escolher exatamente quais arquivos (ou partes deles) serão commitados
- separar mudanças diferentes em commits diferentes
- evitar salvar coisas por engano
- criar commits limpos, organizados e fáceis de entender

Você pode ver o que está na staging area usando:

```
git status
```

Exemplo:

```
Changes to be committed:  
new file:  script.py
```

Isso significa que **script.py** já está na staging area e será incluído no próximo commit.

2. Estados dos arquivos no Git

No Git, cada arquivo do projeto pode estar em um destes estados. Entender esses estados é essencial para dominar a staging area e os commits avançados.

2.1. **untracked** — não rastreado

O arquivo existe no diretório, mas o Git ainda não o acompanha.

```
git status
```

Saída:

```
Untracked files:
  script.py
```

Isso significa que `script.py` ainda não faz parte do repositório.

2.2. modified — modificado, mas não preparado

O arquivo foi alterado, mas ainda não está na staging area.

```
git status
```

Saída:

```
Changes not staged for commit:
  modified:   app.py
```

O Git detectou mudanças, mas você ainda não decidiu se elas vão para o próximo commit.

2.3. staged — preparado para commit

O arquivo foi adicionado à staging area e será incluído no próximo commit.

```
git add app.py
git status
```

Saída:

```
Changes to be committed:
  modified:   app.py
```

Agora o Git sabe exatamente qual versão de `app.py` você quer registrar.

2.4. committed — registrado no histórico

O arquivo foi salvo no repositório Git e não há mudanças pendentes.

```
git commit -m "Atualiza app"
git status
```

Saída:

```
On branch main
nothing to commit, working tree clean
```

Isso indica que tudo está sincronizado entre o **working directory**, a **staging area** e o **repositório**.

+ 3. Adicionando mudanças de forma avançada

✓ 3.1. Adicionar um arquivo específico

```
git add arquivo.py
```

Saída:

```
Changes to be committed:
  new file:   arquivo.py
```

✓ 3.2. Adicionar todos os arquivos modificados

```
git add .
```

Saída:

```
Changes to be committed:
  modified:   app.py
  new file:   utils.py
  deleted:    old_config.json
```

✓ 3.3. Adicionar apenas parte de um arquivo (interativo)

```
git add -p arquivo.py
```

Saída típica:

```
diff --git a/arquivo.py b/arquivo.py
@@ -1,4 +1,7 @@
  def soma(a, b):
      return a + b

+def sub(a, b):
+    return a - b

Stage this hunk [y,n,q,a,d,j,J,g/,e,?]?
```

✓ 3.4. Ver o que está na staging area

```
git diff --staged
```

✓ 3.5. Remover algo da staging area

```
git restore --staged arquivo.py
```

Saída:

```
Changes not staged for commit:
  modified:   arquivo.py
```



4. Commits avançados

✓ 4.1. Editar o último commit (--amend)

```
git commit --amend
```

Esse comando permite editar o último commit, seja para ajustar a mensagem ou incluir mudanças que você esqueceu de adicionar.

Saída típica:

```
[main 3f2a1b7] Adiciona função soma e utilidades  
2 files changed, 10 insertions(+)
```

✓ 4.2. Commit com título + descrição

Você pode escrever uma mensagem de commit com título e corpo, útil para explicar melhor mudanças mais complexas:

```
git commit -m "Implementa login" -m "Adiciona validação de senha."
```

O primeiro `-m` é o título; o segundo é a descrição.

✓ 4.3. Criar commits vazios

Commits vazios são úteis para marcar eventos, testar pipelines ou registrar checkpoints:

```
git commit --allow-empty -m
```

5. Ver diferenças antes de commitar

✓ Working directory (mudanças ainda não adicionadas)

Mostra o que mudou nos arquivos **antes** de entrar na staging area:

```
git diff
```

Exemplo de saída:

```
+def sub(a, b):  
+     return a - b
```

✓ Staging area (mudanças que serão commitadas)

Mostra exatamente o que já está preparado para o próximo commit:

```
git diff
```

6. Remover arquivos da staging area

Use este comando quando você adicionou algo por engano à staging area e quer **desfazer o git add**, mas **sem perder as alterações** feitas no arquivo:

```
git restore --staged arquivo.py
```

Saída:

```
Changes not staged for commit:
  modified:   arquivo.py
```

Isso significa que o arquivo voltou ao estado modified, fora da staging area, mas suas mudanças continuam no working directory.

7. Desfazendo commits (com segurança)

✓ 7.1. Desfazer um commit criando outro (seguro)

```
git revert <hash>
```

O revert cria um novo commit que desfaz as mudanças do commit original, preservando todo o histórico.

É a forma segura de desfazer algo, especialmente em repositórios compartilhados.

Saída típica:

```
[main 7f8e9d1] Revert "Adiciona função soma"
```

✓ 7.2. Voltar no tempo apagando commits (perigoso)

```
git reset --hard <hash>
```

O reset --hard move o ponteiro do branch para um commit anterior e apaga todos os commits posteriores, além de descartar mudanças no working directory.

É poderoso, mas perigoso — use apenas quando tiver certeza absoluta.

Saída:

```
HEAD is now at 0f9e8d7 Cria arquivo inicial
```

8. Resumo do fluxo avançado

<code>git status</code>	→ veja o estado dos arquivos
<code>git diff</code>	→ veja o que mudou no working directory
<code>git add -p</code>	→ adicione apenas trechos específicos
<code>git commit -m "msg"</code>	→ crie commits claros e organizados
<code>git log --oneline</code>	→ revise o histórico rapidamente
<code>git revert <hash></code>	→ desfça commits com segurança

Fim do Guia 2 — Staging Area e Commits Avançados

Agora você entende:

- como funciona a staging area
- como escolher o que entra no commit
- como adicionar trechos específicos (`git add -p`)
- como editar commits (`--amend`)
- como ver diferenças (`git diff`)
- como desfazer mudanças com segurança (`git revert`)
- como evitar reescrever histórico acidentalmente
- como trabalhar com commits de forma profissional

Esse conhecimento é essencial para manter um histórico limpo, organizado e fácil de entender.
